

UOS - Capstone

Week 4 - AutoEncoder & Latent Space

박상훈 ¹

2026학년도 2학기

¹Gwanak Lab Co., Ltd., Korea
E. sanghun.park@gwanaklab.co.kr

지난 시간 리뷰

Tabular AutoEncoder

Latent Space 실습

Variational AutoEncoder

지난 시간 리뷰

- MNIST 데이터셋에 대해 **AutoEncoder**를 이용한 압축(Encoder) / 복원(Decoder) 실습을 진행했습니다.
- 학습된 **Latent Space**를 **PCA**와 **KDE**를 이용해 시각화해봤습니다.

이번 시간에는 **Tabular Data**로 같은 흐름(전처리 → AutoEncoder → Latent 시각화)을 진행하고,
마지막에는 **VAE**까지 직접 구현해봅니다.

Tabular AutoEncoder

문제 1. Tabular 데이터 전처리

문제 1

Tabular Dataset에 대해서 전처리를 진행해보시다.

- 연속형(Continuous) 변수: Standard Transformation과 Quantile Transformation으로 각각 전처리하고 결과를 비교해보세요.
- 범주형(Categorical) 변수: Ordinal Encoding과 One-hot Encoding을 각각 적용해보세요.

문제 2. AutoEncoder 학습 및 전처리 비교

문제 2

전처리한 데이터에 대해서 AutoEncoder 학습을 진행해보시다.

- 문제 1에서 만든 여러 전처리 조합(Standard / Quantile × Ordinal / One-hot)으로 각각 AutoEncoder를 학습해보세요.
- 전처리 방법에 따라 학습 과정과 복원 결과가 어떻게 달라지는지 비교해보세요.

문제 3

얼마나 잘 복원되었는지 어떻게 판단할 수 있을까요?

- 각 컬럼(변수)별로 원본과 복원 데이터의 분포를 히스토그램으로 비교해보세요.
- **GMM(Gaussian Mixture Model)**을 이용해 원본/복원 분포를 근사하고, 두 분포가 얼마나 유사한지 확인해보세요.
- **KL-Divergence**는 무엇이고 어떻게 사용할 수 있을까요?
- 연속형/범주형 분포의 차이를 확인할 수 있는 방법을 하나씩 찾아서 확인해보세요

Latent Space 실습

문제 4. Tabular Latent Space 시각화

문제 4

이미지의 Latent를 시각화했던 것의 연장선으로, Tabular 데이터의 Latent도 시각화해봅시다.

- Encoder를 통과시켜 얻은 Tabular latent vector를 **PCA**로 2차원에 투영해보세요.
- 지난 시간에 사용한 **KDE**도 함께 적용해서 latent의 분포 형태를 확인해보세요.

문제 5

학습 때 보지 않은 새로운 데이터를 생성하고 싶다면 어떻게 할 수 있을까요?

- 임의의 학습 데이터 두 개를 골라 각각의 latent $\mathbf{z}_1, \mathbf{z}_2$ 를 구하세요.
- 두 latent의 **1:1 내분점** $\mathbf{z} = 0.5 \mathbf{z}_1 + 0.5 \mathbf{z}_2$ 를 구하고, Decoder에 넣어 결과를 확인해보세요.
- 이미지 데이터와 Tabular 데이터 모두에 대해 진행해보세요.

문제 6

이번에는 내분점을 두 점 중 하나에 매우 가깝게 만들어봅시다.

- 두 latent의 **1:9 내분점** $\mathbf{z} = 0.1 \mathbf{z}_1 + 0.9 \mathbf{z}_2$ (또는 반대 비율)을 구해보세요.
- 문제 5와 동일하게 Decoder에 넣어 복원된 이미지/데이터를 확인해보세요.

문제 7

문제 5, 6의 결과를 비교하며 다음을 정리해보세요.

1. 내분 비율이 달라질 때 복원 결과는 어떻게 변화하나요?
2. 이미지 데이터와 Tabular 데이터 중 어느 쪽에서 변화가 더 뚜렷하게 나타나나요?
3. 이 결과가 Latent space의 어떤 성질을 보여준다고 생각하나요?

Variational AutoEncoder

문제 8

이제 VAE를 구현해봅시다. 먼저 AutoEncoder 코드에서 어떤 부분을 추가/제거해야 할지 정리해보세요.

- Encoder의 출력이 어떻게 바뀌어야 할까요? (하나의 $\mathbf{z} \rightarrow ?$)
- 새로 추가해야 하는 계산(함수)은 무엇인가요?
- Loss function에 추가로 필요한 항은 무엇인가요?

문제 9. Reparameterization 없이 구현해보기

문제 9

Reparameterization trick을 사용하지 않으면 어떤 문제가 생길까요? 직접 확인해봅시다.

- 기존 코드의 $z = \mu + \text{std} * \epsilon$ 부분을, numpy를 이용한 직접 샘플링(예: `np.random.normal(mu, std)`)으로 바꿔보세요.
- 동일한 조건으로 학습을 진행해보고 어떤 문제가 발생하는지 관찰해보세요. (학습이 정상적으로 진행되나요? 에러가 발생하나요?)
- 왜 이런 문제가 생기는지 이유를 생각해보세요.

문제 10

학습된 VAE를 이용해 새로운 이미지/데이터를 생성해봅시다.

- Prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 에서 $\mathbf{z}$ 를 직접 샘플링하고, Decoder에 넣어 이미지/Tabular 데이터를 생성해보세요.
- 앞선 문제들의 AutoEncoder 실험과 비교했을 때 어떤 차이가 있는지 확인해보세요.